UPTEC XX XXXXX

Degree project 30 credits

May , 2026

Heterogeneous Multi-Robot Collaboration Using Language Models

Exploring different sizes and architectures

Rohit Joseph Mamutil

Master's Programme in Embedded Systems

Heterogeneous Multi-Robot Collaboration Using Language Models

Rohit Joseph Mamutil

Abstract

Heterogeneous warehouse fleets typically include robots with different roles, such as carrier vehicles and picker agents, that must coordinate task choice, role synchronization, and recharging while sharing shelves, corridors, and charging stations. This creates a discrete multi-agent decision problem in which throughput depends not only on motion planning, but also on when agents should commit to tasks, wait for support, or allocate time to battery recovery.

This thesis evaluates whether off-the-shelf language models (LMs) can support high-level decision-making for heterogeneous multi-robot collaboration in the Battery-TA-RWARE task-assignment warehouse environment. The control loop summarizes the current environment state, queries an LM for a grounded macro-action, and validates the proposed action against environment-defined feasibility constraints before execution. The experiments compare model size, planning architecture, and prompt representation, using delivery throughput, LM call efficiency, and observed failure behaviour as the main evaluation criteria.

The available data show that, in the shelf-delivery experiments, shared-context planning produced more deliveries than centralized planning and natural-language prompting produced more deliveries than JavaScript Object Notation (JSON) prompting. In the call-aware experiments, a mid-sized local model (llama3.2:3b) produced the highest delivery total among the completed natural-language runs, while the completed JSON runs produced lower throughput and higher call cost than their natural-language counterparts. Across the evaluated settings, the control loop operated through grounded action identifiers, candidate shaping, and explicit validation before execution.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Xuezhi Niu Subject reader: Didem Gürdür Broo

Examiner: Ramanathan Thinniyam Srinivasan

Sammanfattning

I heterogena robotflottor i lager används ofta olika robottyper (t.ex. bärande automated guided vehicles (AGVs) och lastningsagenter) som måste samordna sina handlingar för att uppnå hög genomströmning samtidigt som säkerhetskrav, såsom batterinivåer, uppfylls. Att konstruera styrsystem för sådana multi-robot systems (MRSs) är svårt på grund av stora diskreta handlingsrymder, glesa belöningssignaler och behovet av robust koordination.

Detta examensarbete undersöker om färdigtränade language models (LMs) kan användas som beslutsfattare på hög nivå för samarbete mellan heterogena robotar i en lager-miljö med uppgiftsallokering och batteridynamik (Battery-TA-RWARE). Metoden sammanfattar det multi-agentiska tillståndet i naturligt språk, låter en LM välja en handling per agent och validerar sedan handlingarna mot miljöns tillåtna handlingsmasker.

Arbetet levererar en första utvärderingspipeline för LM-styrning av multi-agentisk lagerlogistik samt preliminära resultat som visar att en enkel språkbaserad LM-policy presterar avsevärt sämre än en domänheuristik, vilket motiverar fortsatt arbete med bättre struktur, grounding och koordination.

Acknowledgement I would like to thank my supervisor, Xuezhi Niu, for continuous guidance and constructive feedback throughout this thesis work. I also thank my reviewer, Didem Gürdür Broo, and my examiner, Ramanathan Thinniyam Srinivasan, for their input on the direction and quality of the work. Finally, I am grateful to the developers and maintainers of the open-source environments and tools used in this project.

Contents

Acronyms

AAMAS Autonomous Agents and Multiagent Systems. 5

AGV automated guided vehicle. ii, 1, 6, 8, 9, 13, 15

JSON JavaScript Object Notation. i, 3–5, 11, 13–15, 18–24

LM language model. i, ii, 1–3, 5–13, 15, 17, 19–25, 29

MARL multi-agent reinforcement learning. 2, 5, 25

MAS multi-agent system. 5, 6

MRS multi-robot system. ii, 2, 5, 6

PPO proximal policy optimization. 25

TA-RWARE task-assignment robotic warehouse environment. 8

1 Introduction

Accounts of language use and language evolution have often linked language to coordination, cooperation, and shared intentionality in social groups [6, 22, 8]. In these accounts, language is not only a means of describing the world, but also a means of aligning intentions, communicating plans, and organizing joint activity. This perspective motivates the present thesis: if language is closely associated with coordination in human settings, then language-capable models may also be worth studying as decision components in multi-agent systems, provided that the setting is concrete enough to test that idea under operational constraints.

Warehouse automation provides a concrete setting in which these coordination problems can be studied under operational constraints. Large-scale robotic warehouses became a prominent example of multi-robot coordination through systems such as Kiva, where many autonomous drive units transport movable inventory pods to human pick stations [23]. The key operational idea in such goods-to-person fulfilment systems is that mobile robots bring storage units to stationary workers, rather than requiring workers to walk through storage aisles. This makes warehouse robotics a useful real-world domain for studying task allocation, congestion, synchronization, and fleet-level coordination among many autonomous vehicles [3].

The environment studied in this thesis follows this broader goods-to-person warehouse tradition, but uses the heterogeneous Battery-TA-RWARE setting, which adapts the Task-Assignment Robotic Warehouse environment introduced by Krnjaic et al. [14, 13]. TA-RWARE is motivated by Quicktron QuickBin-style goods-to-person systems, in which storage retrieval and transport can be split across different robot types. This differs from Kiva-style systems, where a drive unit can lift and transport an inventory pod by itself. In TA-RWARE, carrier automated guided vehicles (AGVs) transport storage mediums such as shelves or totes, while picker agents support the loading and unloading operations required to place those storage mediums onto AGVs. The human picker at the delivery station is not part of the simulated multi-agent system; in TA-RWARE, the picker role denotes a picking robot that lifts storage mediums onto AGVs, while the human worker remains outside the simulated robot team. This terminology is important because the thesis studies coordination between robot roles, not human-robot picking at the workstation.

The resulting decision problem is therefore not only where agents should move, but also which shelf should be served next, when an AGV should wait for a picker robot, when a carried shelf should be delivered or returned, and when battery recovery should take priority over immediate task progress. This role separation introduces interdependent tasks: AGVs and picker robots must select compatible storage targets and arrive

with sufficient temporal alignment for pickup operations to succeed [14]. This is a task-level coordination problem as well as a motion problem: multi-robot task allocation concerns which agents should perform which tasks, while multi-agent path finding concerns collision-free movement to assigned goals [9, 21]. Warehouse settings further make this distinction visible because agents are repeatedly assigned new goals while operating in shared space [15]. Productive coordination therefore requires role synchronization, task commitment, stall recovery, and adaptation to changing objectives. For example, an AGV may reach a shelf and still fail to make progress if a picker robot is not synchronized in time; repeated lack of position change can indicate interaction conflicts in the environment and motivate trying a different action; and a controller may need to shift behavior when the objective changes from maximizing deliveries alone to also reducing unnecessary model calls or respecting battery-related constraints. The thesis therefore studies the higher-level decision layer above low-level motion: selecting grounded macro actions that determine what each agent should do next under the current task, coordination, and battery conditions.

Multi-robot systems (MRSs) provide useful background for this problem. Early surveys emphasized group architecture, resource conflicts, and learning [5], while later overviews highlighted communication, task allocation/control, and motion coordination as recurring themes [2]. In the warehouse setting of this thesis, these themes translate into concrete questions about assigning requested shelves, synchronizing heterogeneous roles at shelf locations, handling conflicts in shared corridors, and respecting energy-related constraints without stalling delivery progress.

Many approaches address such coordination problems using optimization, hand-engineered heuristics, or multi-agent reinforcement learning (MARL). Optimization and heuristic methods can be effective, but they often require substantial domain-specific design and retuning when the operating conditions or objective formulation change. MARL can learn coordinated policies, but training remains expensive, strongly dependent on reward design and experimental protocol, and often difficult to adapt when the environment, agent mix, or optimization target changes. Benchmarking studies further show that conclusions in multi-agent learning are highly task- and metric-dependent, which makes careful experimental framing important [19]. Recent work in heterogeneous multi-robot settings has also explored bio-inspired *symbiosis* framings and reward shaping strategies to stabilize cooperation under centralized training and decentralized execution [17, 18].

Language models (LMs) offer a complementary alternative in this space. In this thesis, LMs refer to prompted pre-trained text-generation models that are used as high-level decision components, rather than as low-level controllers or task-specific policies trained directly inside the warehouse environment. Instead of learning a task-specific policy from scratch, an LM can be prompted with a structured summary of the current

environment state and asked to choose a grounded macro action, meaning a validated high-level target choice such as a shelf, goal station, or charging station. In this thesis, a high-level decision means selecting a discrete environment action identifier, such as moving toward a shelf, a goal station, or a charging station, rather than generating free-form robot commands. Recent robotics work has used LMs together with explicit grounding and feasibility checks so that language reasoning remains tied to validated actions instead of unconstrained text outputs [1]. This makes LMs a plausible object of study for warehouse coordination, but it also raises a concrete question: can they support productive heterogeneous coordination under discrete action constraints, limited context, and changing objectives?

1.1 Research Questions

The thesis is guided by the following main research question:

> **Main research question:** To what extent can prompted LMs support grounded high-level decision-making for heterogeneous multi-robot collaboration in a constrained warehouse environment?

To support a systematic investigation of the main research question, the study considers the following sub-questions: In this thesis, *centralized planning* means that one prompt receives the shared state and returns decisions for all agents that currently need planning, whereas *shared-context per-agent planning* means that agents are planned one at a time, but each prompt still contains a shared summary of the global warehouse state.

1. **RQ1 (Model size):** How does language-model size affect collaborative task performance in the warehouse setting, measured by delivery throughput, language-model calls per delivery, and observed failure behaviour across scenarios?
2. **RQ2 (Planning architecture):** How do centralized and shared-context per-agent planning architectures differ in collaborative task performance, measured by delivery throughput, robustness across scenarios with different levels of role-interaction pressure, and the frequency of non-productive runs or collaboration failures?
3. **RQ3 (Prompt representation):** How do natural-language and JavaScript Object Notation (JSON) prompt representations differ in collaborative task performance, measured by delivery throughput, call efficiency, and the consistency of valid, productive collaborative decisions?

In this thesis, these questions are evaluated primarily through delivery throughput, language-model call efficiency, and observed failure behaviour. Depending on the experiment family, the analysis also considers robustness across scenarios, the prevalence of zero-delivery or long no-delivery runs, and whether productive coordination is maintained when the objective changes.

1.2 Contributions

The current thesis work contributes:

- A grounded evaluation setting for studying language-model-based high-level control in heterogeneous warehouse coordination, where proposed decisions are translated into discrete environment actions and checked against explicit feasibility constraints rather than executed as free-form outputs.
- Comparative evidence on three design variables relevant to the research questions: model size, planning architecture, and prompt representation. In particular, the experiments provide a basis for assessing how these choices affect delivery throughput, call efficiency, robustness across scenarios, and the prevalence of non-productive behaviour.
- Empirical insights into the conditions under which the approach remains productive. The current results indicate that shared-context per-agent planning is more promising than centralized planning in the evaluated shelf-delivery setting, that natural-language prompting is more effective than JSON prompting in the completed comparisons, and that language-model-based control depends strongly on grounding and rule-based safeguards to avoid systematic failure modes such as no-delivery tails, stale waiting loops, and other coordination breakdowns.

1.3 Report Outline

Section 2 provides background on multi-robot coordination, warehouse task assignment, and LM-based planning. Section 3 describes the environment, the proposed LM control loop, the experimental design, and the main delimitations of the study. Section 4 presents the current experimental results together with their interpretation. Finally, Section 5 concludes and outlines future work.

2 Background and Related Work

This chapter groups prior work along the dimensions that matter most for the thesis design: coordination structure in multi-robot systems, the warehouse task-assignment setting used as the testbed, and grounded language-model planning under explicit action constraints. Its purpose is to show how these lines of work motivate the chosen experimental design variables: planning architecture, information sharing, decision timing, prompt structure, and model size.

2.1 Multi-Robot Systems and Coordination

MRSs study how multiple embodied agents share resources, divide work, communicate, and avoid conflicts while pursuing a common objective. The broader field of multi-agent system (MAS) can also include purely software agents, whereas MRSs typically emphasize physical robots operating under motion, sensing, and actuation constraints. Classical overviews of cooperative robotics highlight recurring concerns such as group architectures, resource conflicts, and learning [5]. Survey treatments vary in terminology and scope (e.g., differences in how MAS and MRS are framed, and how communication and coordination taxonomies are defined) [25, 2]. In heterogeneous teams, these concerns intensify because roles and capabilities differ, and success often requires explicit synchronization, such as meeting at the same physical location at the same time [20]. This is directly relevant to the thesis, where AGVs and pickers cannot complete the full warehouse cycle independently and therefore depend on timely coordination under shared-space and battery constraints.

One useful way to organize prior work is by the aspects of coordination it emphasizes. Some work focuses on *coordination structure*, for example whether decisions are made centrally or through repeated local coordination. Other work focuses on *information sharing*, such as the role of explicit communication versus coordination through shared state or environment cues. Another recurring distinction is *decision timing*: whether coordination is fixed in advance, recomputed online, or revised in response to new state information. These dimensions map directly to the thesis design choices, especially the comparison between centralized and shared-context planning, the use of shared state summaries in prompts, and the use of step-synchronous conditional replanning rather than a fully fixed policy.

In this thesis, *collaboration* refers to interdependent task execution among heterogeneous roles, where agents must coordinate to complete an operation (e.g., a load/unload interaction). *Coordination* refers to mechanisms that prevent conflicts and deadlocks

(e.g., collision avoidance and congestion handling), and *cooperation* refers to pursuing a shared objective even when agents may act more independently. These terms are not used consistently across communities, and the scope of MAS often extends beyond embodied robotics [12]. To reduce ambiguity, the thesis primarily uses MRS when referring to embodied robots and uses MAS to denote the broader class of multi-agent systems.

Coordination strategies can be described as *static* (offline, convention-based) or *dynamic* (online, reactive), depending on whether coordination rules are established prior to execution or synthesized during execution from sensed/communicated information [25]. Communication can likewise be categorized as *implicit* (information inferred via sensing or environment traces) versus *explicit* (direct message passing), trading off scalability against accuracy and bandwidth [25]. For example, Zhang et al. study coordinated MARL under communication constraints, illustrating the broader trade-off between tighter coordination and limited information bandwidth in multi-agent decision-making [27].

Planning is a central lens for collaboration: multi-robot planning is often discussed as task planning (who does what) and motion planning (how to move), with optimal formulations typically being NP-hard and thus motivating approximate methods and heuristics [25]. In particular, multi-robot task planning includes task decomposition (splitting a mission into subtasks) and task allocation (assigning subtasks to agents), with much of the literature focusing on allocation due to tractability and benchmarking convenience [25]. This observation aligns with the thesis hypothesis: LMs may be useful as a structured decision module for decomposition and allocation when paired with explicit grounding, feasibility constraints, and environment feedback.

2.2 Warehouse Task Assignment Environments

This thesis uses Battery-TA-RWARE, an adaptation of the Task-Assignment Robotic Warehouse environment introduced in [14]. It is used here as a structured warehouse testbed in which repeated shelf-delivery cycles create explicit coordination pressure across heterogeneous roles. The environment models two heterogeneous agent types:

- **AGVs:** carrier agents that transport shelves between storage locations and goal stations.
- **Pickers:** support agents that must meet AGVs at shelf locations for loading and unloading.

Battery-TA-RWARE extends the task-assignment setting with battery depletion and charging stations, forcing agents to balance throughput with energy safety. Low-level movement is handled by environment dynamics (including A* path planning), while the controller selects discrete targets representing shelves, goals, or charging stations. This makes the environment especially useful for the thesis because it isolates the task-level coordination problem: which target should be chosen next, under what shared-state conditions, and with what timing relative to other agents.

2.3 Language-Model-Based Planning and Grounding

LMs can produce coherent natural-language plans, but robot control requires converting language into *grounded*, *feasible* actions. One influential pattern is to constrain the LM to select among validated, environment-defined options. For example, SayCan combines language-level reasoning with an affordance model to filter actions that are executable in the current state [1]. Other work frames LMs as planners that can propose action sequences given a symbolic interface to the environment, often coupled with explicit checking and replanning [10]. In language-only settings, prompting patterns such as interleaving reasoning and actions have been studied to improve decision quality under tool or environment feedback [26]. Recent multi-robot systems work explores *multi-agent* prompting where each robot maintains a separate dialog context and coordination emerges through structured communication plus environment feedback (e.g., collision checking and in-context plan revision) [16]. Recent work argues that closed-loop state feedback and a separation between high-level planning and low-level control can reduce hallucinated actions and improve long-horizon goal satisfaction in robot task planning with LMs [4]. In multi-robot task planning contexts, prior thesis work also emphasizes the need for a symbolic abstraction layer that translates sensor/state representations into language-level predicates that LMs can reliably consume [11].

These ideas motivate the design in this thesis: use a prompt that provides a compact but explicit mapping from environment state to action identifiers, require outputs to be parseable into discrete actions, and validate them against the environment's current action constraints.

2.4 Why Study Model Size and Prompt Design?

In applied robotics and embedded contexts, compute, memory, energy, and latency constraints may favour smaller models for real-time deployment [7]. However, smaller models may require more structured prompting, tighter candidate sets, and stronger

validation to reach acceptable performance. Recent robotics work explicitly motivates prompt design as a lever for compensating limited model capacity and improving decision quality in constrained deployments [24, 16]. In the warehouse setting of this thesis, these choices are not treated as generic language-model properties. They are studied because they are expected to affect whether generated actions remain feasible, whether coordination remains productive under shared constraints, and how often the controller falls into invalid or non-productive behaviour. By evaluating multiple model sizes and prompt variants under the same environment and metrics, the thesis aims to clarify which components (model capacity, prompt structure, validation) most strongly affect feasibility, coordination quality, and throughput.

3 Methodology

3.1 Overview

This chapter serves two roles. It describes the implemented system, but it also justifies the empirical design used to answer the research questions. The thesis is therefore structured as a comparative experimental study: the same warehouse environment, evaluation metrics, and scenario catalogue are reused while selected design variables are changed in order to study their effect on collaborative task performance.

This design is appropriate for the thesis because the research questions are comparative rather than purely descriptive. RQ1 asks how model size affects performance, RQ2 asks how planning architecture affects performance, and RQ3 asks how prompt representation affects performance. These questions require the report to make explicit what is fixed, what is varied, and how the experiments are structured across the objective families.

The thesis uses an empirical, experimental methodology:

1. Implement an LM-driven control loop for a simulated heterogeneous warehouse environment.
2. Define the main comparison axes and evaluation metrics.
3. Run controlled experiments (fixed environment ids and seeds) across multiple LM models, planning architectures, and prompt variants.
4. Analyze throughput, safety, and collaborative task outcomes as well as qualitative failure modes.

What is fixed and what is varied. Across the main experiment families, the environment, action space, scenario catalogue, evaluation metrics, grounding/validation scheme, and local-inference setup are held as stable as practical. The principal variables are model size, planning architecture, prompt representation, and objective family. The report does not use an external heuristic baseline in the current comparison set, so the reported comparisons are made within the language-model-based control framework.

Interpretation and controls. The experiments are controlled in the sense that scenario ids, seeds, and runner configurations are recorded and reused within each family. The surrounding validation layer filters infeasible outputs before execution, and the same comparison axes are used across the objective families.

3.2 Environment: Battery-TA-RWARE

Battery-TA-RWARE is a discrete-time, grid-based warehouse simulation derived from task-assignment robotic warehouse environment (TA-RWARE) [14]. At any time, a set of shelves is requested. Carrier AGVs must transport requested shelves to goal locations and then return them to storage. Picker agents are required to meet AGVs at shelf locations for load and unload interactions, creating explicit heterogeneous coordination constraints. Agents additionally have a battery state that decreases with movement and interactions; charging stations are available at fixed locations.

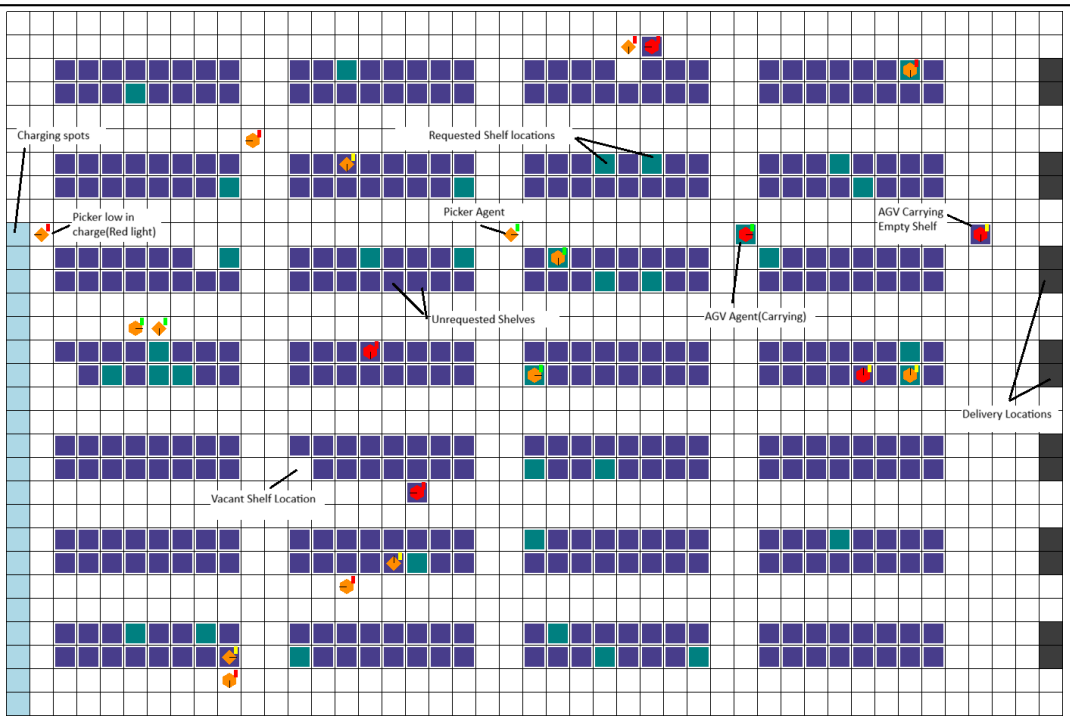

Figure 1 Illustration of the task-assignment warehouse setting used in this thesis.

The environment exposes a discrete action space where each action corresponds to selecting a target location:

- **Shelf actions:** choose a shelf location (e.g., to pick up or return a shelf).
- **Goal actions:** choose a delivery station (primarily relevant when an AGV carries a requested shelf).
- **Charging actions:** choose a charging station to recover battery.

Low-level motion toward the selected target is handled internally by the environment (A* path planning and collision resolution), while the controller is evaluated on the quality of these high-level decisions.

3.3 Language-Model Control Loop

The LM policy is implemented as a text-in / discrete-action-out controller:

1. **State summarization:** extract salient features of the global state (requested shelves and their action ids, per-agent position/target/battery/carrying status, charging occupancy).

2. **Candidate action set:** compute a valid-action mask from the environment and derive the prompt-visible candidate set from those feasible actions. These candidates are presented as compact role-relevant action rows with action ids and guidance fields such as distance, tags, and charging-related instructions.

3. **Prompting:** generate a per-agent prompt describing the agent role and the current state, including explicit mappings from requested objects to action ids.

4. **Querying:** send the prompt to a local inference server (Ollama) and obtain a text response.

5. **Parsing:** parse an integer action id (and optionally an action hold duration) from the response.

6. **Validation:** validate the parsed action id against the current valid-action mask. If the proposed action is missing or invalid, the controller discards it and falls back to an idle action for that step; in the action-persistence setting, an invalid persisted action clears the hold and forces a new query.

7. **Execution:** apply the validated joint action in the environment for one step (or hold the action for multiple steps, depending on configuration).

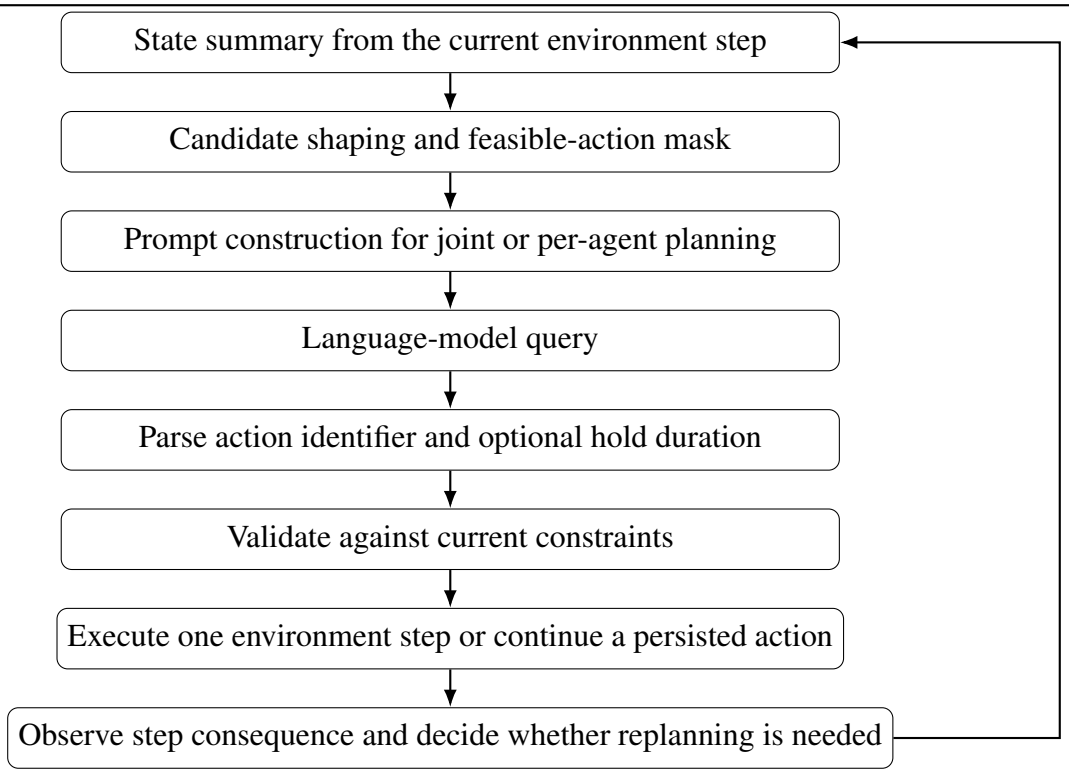

Figure 2 Schematic of the language-model control loop used in the experiments.

The controller is best described as a discrete-time, step-synchronous supervisory loop with conditional replanning. The environment advances one simulation step at a time, and at each step the controller reevaluates agent state and action validity. However, this does not imply that every agent is queried at every step. Instead, the experiments use two related control schemes.

- **Step-synchronous replanning with environment-owned continuation:** agents that are currently available for planning may receive a new high-level action, while busy agents are not replanned and continue their current task until the environment releases them.
- **Step-synchronous replanning with action persistence and conditional override:** accepted actions may remain active for multiple future steps, so agents are not re-queried while the persisted action remains valid. Replanning is triggered when a persisted action expires, becomes invalid, or when the controller decides intervention is needed, including selected busy-agent cases.

This design sits between two extremes. A fully scheduled scheme that queries every

agent at every step would generate many unnecessary LM calls, while a purely event-triggered scheme is less natural in this setting because useful triggers such as blocked progress, invalid persisted actions, expiring commitments, or changing support needs are only known after the system state is checked. The adopted design therefore combines regular per-step observability with selective replanning. This is also consistent with practical supervisory control in warehouse systems, where the fleet is typically checked at regular intervals, ongoing tasks continue by default, and new task-level decisions are issued only when the current situation justifies intervention.

Two design choices are central for feasibility:

- **Grounding via action ids:** the LM does not output coordinates or free-form text commands; instead it selects from discrete action ids that are already defined by the environment.
- **Hard validation:** action masks enforce constraints such as invalid goals for picker agents and busy-state restrictions. Invalid LM outputs are never executed.

The reported behaviour is therefore the behaviour of the complete control framework: prompt design, model output, candidate shaping, validation logic, and replanning policy act together inside the same supervisory loop.

3.4 Experiment Families and Objectives

The implemented experiments are grouped into three objective families. Across these families, the common control framework is the grounded language-model loop described above: step-synchronous execution, conditional replanning, explicit action validation, and action persistence when configured. Across all three objective families, the same controlled variables are intended: planning architecture, prompt representation, model size, scenario catalogue, and evaluation metrics. What changes between the families is the objective function.

Objective 1: maximize total shelf deliveries. The first family fixes the operational objective to raw shelf-delivery throughput. This family provides the delivery-only baseline against which the other objective formulations are interpreted.

Objective 2: optimize LM calls while maintaining delivery throughput. The second family changes the objective from delivery-only to a joint objective: increase

shelf deliveries while reducing unnecessary LM queries. It primarily supports RQ1 and the call-efficiency analysis while keeping the same control framework and the same comparison axes as Objective 1. In this family, the output interface includes a Steps field so that the selected action can remain active for multiple steps when that serves the call-aware objective.

Objective 3: battery-aware objective. A third family is defined for explicitly battery-oriented evaluation. Its role is to extend the same framework toward constraint-aware behaviour beyond throughput and call efficiency.

3.5 Planning Architectures

The experiments compare two distinct ways of integrating the LM into the decision loop.

Centralized planning. In the centralized architecture, one LM call plans all agents that currently need a decision. This design exposes the full global state and all relevant candidate actions in one prompt. Its intended advantage is explicit joint coordination. Its main risk is prompt overload: as the number of agents and candidate actions increases, the prompt becomes large and more difficult to parse reliably.

Shared-context per-agent planning. In the shared-context architecture, the LM plans one agent at a time. Each prompt still includes global information, but the final decision concerns only the current agent. This reduces the output burden and simplifies parsing, at the cost of requiring coordination to emerge from repeated local decisions rather than from one explicit joint plan. In the thesis evaluation, this architecture is the main vehicle for comparing prompt forms and model sizes under a common decision structure.

3.6 Prompt Structures

The prompt comparison in this thesis varies along two axes:

- **Planning scope:** centralized joint planning versus per-agent planning with shared context.

- **Output structure:** natural-language responses versus JSON responses.

The prompts include:

- explicit requested-shelf to action-id mappings,
- role-specific rules for AGVs and pickers,
- support relations such as waiting-for-picker context,
- battery-state abstractions,
- candidate action filtering or prioritization,
- and, in Objective 2, an explicit Steps field as part of the call-aware output interface for reducing unnecessary LM calls.

3.7 Comparison Axes

The main comparison axes are:

- model size,
- planning architecture (centralized versus shared-context per-agent),
- prompt form (natural language versus JSON),
- and objective formulation (shelf-delivery only, call-aware shelf delivery, and battery-aware control) under the same grounded control framework.

3.8 Evaluation Metrics

The main metric is delivery throughput, expressed as a pick rate (deliveries per hour) derived from the number of delivered shelves and the simulated episode duration. Additional metrics are collected to capture coordination and safety:

- total shelf deliveries,
- number of clashes and deadlocks/stuck events,

- distance travelled by each agent type,
- episode return (environment reward),
- episode length and inference speed/latency (when measured).

The primary purpose of these metrics is to evaluate whether the language-model-based controller adapts productively when the objective changes. Throughput remains the central outcome measure, since each objective still depends on productive task completion. The additional metrics are used as supporting evidence for how that adaptation occurs in practice, for example whether a configuration maintains delivery performance, reduces unnecessary model calls, or exhibits coordination failures such as clashes, deadlocks, zero-delivery runs, or long no-delivery tails.

3.9 Scenario and Model Matrix

The same scenario catalogue is reused across the experiment families:

- tiny_balanced_1v1,
- small_balanced_2v2,
- medium_balanced_4v4,
- small_agv_heavy_4v2,
- medium_agv_heavy_6v2,
- small_picker_heavy_2v4.

These scenarios were selected to probe different coordination pressures: simple balanced collaboration, increased concurrency, larger warehouses, AGV-heavy support bottlenecks, and picker-heavy underutilization. More specifically, the balanced small scenarios provide lower-pressure comparison points for model and prompt behaviour, the larger balanced scenarios expose increased concurrency and scalability pressure, the AGV-heavy scenarios stress picker-support bottlenecks and waiting dependencies, and the picker-heavy scenarios stress allocation efficiency and underutilization. The scenario set is used to test whether the same model or architecture remains productive when the source of coordination difficulty changes.

The model set spans smaller and larger local models:

- llama3.2:1b,
- llama3.2:3b,
- mistral:7b,
- gemma3:12b,
- phi4,
- qwen2.5:14b.

3.10 Experimental Protocol

The reported experiment sessions were run with fixed environment ids and a fixed seed. This defines a controlled set of comparable runs for the reported objective families.

The progression of the experimental campaign was:

1. Objective 1 shelf-delivery experiments,
2. Objective 2 call-aware shelf-delivery experiments under the same control framework and comparison axes,
3. Objective 3 battery-aware experiments.

Section 4 reports Objective 1 for the shelf-delivery comparisons, Objective 2 for the call-aware comparisons, and Objective 3 for the battery-aware objective. In the current simulation setup, a run can achieve at most 20 deliveries, and the results chapter uses that ceiling to contextualize reported scores.

3.11 Delimitations

To keep the study focused and interpretable, the following delimitations apply:

- The work is evaluated in a simulated warehouse environment rather than on physical robots.
- The controller is responsible only for high-level target selection; low-level navigation and collision handling are performed by the environment.

- The evaluated language models are prompted general-purpose models accessed through a local inference server and are not fine-tuned for this task.
- The reported experiments use fixed seeds and a limited number of completed sessions.
- Completed aggregate results are not yet available for the battery-aware objective family.

3.12 Ethical and Sustainability Considerations

Although the present work is simulation-based, it raises practical issues that remain relevant for real multi-robot deployments. The primary ethical concern is safety and reliability: generated decisions may be infeasible, unstable, or unproductive if they are not constrained by system-side checks. For that reason, the studied control loop does not execute free-form outputs directly. It uses grounded action identifiers, candidate shaping, explicit feasibility validation, and fallback behavior so that unsafe or clearly invalid actions are filtered before execution. The logging of prompts, parsed outputs, executed actions, and observed step consequences also improves traceability, which is important when analyzing failures such as long no-delivery tails, stale waiting loops, or battery-related breakdowns.

From a sustainability perspective, the main trade-off is between model capability and computational cost. Larger language models generally require more inference time and compute, while smaller models may require stronger prompt structure and validation to remain effective. This thesis therefore treats call efficiency, unnecessary replanning, and coordination overhead as practically relevant concerns rather than only implementation details. Even with safeguards, fixed-seed evaluation, limited scenario coverage, and the absence of physical deployment still bound the scope of the reported results.

4 Preliminary Results and Discussion

4.1 Result Scope and Objective Progression

This chapter reports the available results for the three objective families. Objective 1 focuses on maximizing total shelf deliveries. Objective 2 retains shelf delivery as the task outcome, but changes the optimization target to improving delivery while reducing un-

necessary LM calls. Objective 3 is defined as a battery-aware objective, but completed aggregate results are not yet available for that family.

For interpreting the reported numbers, one additional fact is important: in the current simulation setup, 20 deliveries is the maximum achievable total for a run. Reported delivery counts should therefore be read relative to this ceiling. A run with only a few deliveries is not merely weak in absolute terms; it is far below what the environment allows when coordination succeeds.

4.2 Objective 1: Shelf-Delivery Experiments

Objective 1 evaluates planning architecture and prompt form under the shelf-delivery objective.

4.2.1 Planning architecture

The architecture comparison in Objective 1 is summarized in Table 1.

Experiment family	Runs	Total deliveries	Average deliveries/run
Centralized + natural	36	15	0.42
Centralized + JSON	36	21	0.58
Shared-context + natural	36	25	0.69
Shared-context + JSON	36	0	0.00

Table 1 Aggregate outcome of the Objective 1 shelf-delivery experiments across models and scenarios.

Table 2 reports the matched natural-language Objective 1 runs, where the prompt form and objective are the same and the planning architecture differs.

Architecture	Runs	Total deliveries	Average deliveries/run
Centralized + natural	36	15	0.42
Shared-context + natural	36	25	0.69

Table 2 Direct Objective 1 architecture comparison under the matched natural-language setting.

Under this matched setting, shared-context planning produced 25 deliveries in aggregate and 0.69 deliveries per run on average, whereas centralized planning produced 15

deliveries in aggregate and 0.42 deliveries per run on average.

4.2.2 Prompt representation

Objective 1 and Objective 2 each contain matched shared-context runs with natural-language and JSON prompts. Table 3 reports those side-by-side comparisons.

Objective family	Architecture	Prompt form	Runs	Total deliveries
Objective 1	Shared-context	Natural language	36	25
Objective 1	Shared-context	JSON	36	0
Objective 2	Shared-context	Natural language	36	303
Objective 2	Shared-context	JSON	36	53

Table 3 Direct prompt-format comparisons within matched shared-context experiment families.

In Objective 1, the matched shared-context natural-language runs produced 25 deliveries, whereas the matched shared-context JSON runs produced zero. In Objective 2, the matched natural-language shared-context session produced 303 deliveries, whereas the matched JSON session produced 53.

4.3 Objective 2: Call-Aware Shelf-Delivery Experiments

Objective 2 changes the goal from pure delivery throughput to a joint objective: preserve delivery performance while reducing unnecessary LM calls. The output interface for this objective includes both an action identifier and a Steps value, where Steps specifies how long the chosen action should remain active before the next query.

Across 36 runs, the completed Objective 2 natural-language shared-context session produced 303 deliveries, corresponding to an average of 8.42 deliveries per run.

4.3.1 Prompt representation

The completed Objective 2 sessions provide a direct prompt-format comparison within the shared-context experiment family. The natural-language session produced 303 deliveries across 36 runs, corresponding to 8.42 deliveries per run. The completed JSON session produced 53 deliveries across the same number of runs, corresponding to 1.47

deliveries per run. Efficiency measures show the same pattern: the natural-language session required approximately 204.7 LM calls per delivery and 469.5 seconds per delivery, while the JSON session required approximately 1078.4 calls per delivery and 3057.4 seconds per delivery.

The Objective 2 JSON session produced non-zero throughput, with deliveries reported for gemma3:12b and phi4. The natural-language session had 2 zero-delivery runs out of 36, whereas the JSON session had 26 zero-delivery runs out of 36.

4.3.2 Adaptation to the changed objective

Objective 2 reports the available data for the call-aware objective. The completed runs report delivery totals together with call-related metrics through the Steps output and the logged query counts.

4.3.3 Model size

Table 4 reports the completed model-size comparison from Objective 2.

Model	Total deliveries	Average deliveries/scenario	Seconds/delivery
llama3.2:3b	73	12.17	238.5
gemma3:12b	63	10.50	460.2
mistral:7b	59	9.83	530.5
qwen2.5:14b	46	7.67	495.6
llama3.2:1b	36	6.00	371.4
phi4	26	4.33	1091.6

Table 4 Aggregate results for the Objective 2 shared-context natural-language session.

Within the completed natural-language session, llama3.2:3b produced the highest total delivery count and the lowest seconds-per-delivery value among the listed models. In the completed JSON session, gemma3:12b and phi4 produced 26 and 25 deliveries respectively, while both llama3.2 variants produced zero deliveries across all six scenarios.

4.3.4 Call-efficiency outcome

Objective 2 reports both throughput and LM usage. Raw calls-per-delivery were approximately 146.2 for phi4, 158.3 for gemma3:12b, 167.0 for qwen2.5:14b, 167.7 for llama3.2:3b, 287.0 for mistral:7b, and 316.3 for llama3.2:1b.

4.4 Objective 3: Battery-Aware Objective

Objective 3 defines a battery-aware objective. Completed aggregate results are not yet available for this objective family.

4.5 Discussion

4.5.1 RQ1: Model size

The available data for RQ1 come mainly from Objective 2. Table 4 reports the completed model-size comparison for the shared-context natural-language session, and the accompanying text reports the corresponding JSON outcomes where available.

4.5.2 RQ2: Planning architecture

The available data for RQ2 come from the matched Objective 1 natural-language comparison, where shared-context planning produced 25 deliveries and centralized planning produced 15.

4.5.3 RQ3: Prompt representation

The available data for RQ3 come from the matched shared-context prompt-format comparisons in Objective 1 and Objective 2. These comparisons report delivery totals, zero-delivery runs, calls per delivery, and seconds per delivery for natural-language and JSON prompting.

The reported sessions use fixed seeds. Completed aggregate results are not yet available for Objective 3.

5 Conclusions and Future Work

This thesis evaluates LM-driven decision-making for heterogeneous multi-robot collaboration in a task-assignment warehouse setting with battery constraints. The implemented control loop grounds decisions to discrete environment action identifiers, queries an LM using centralized or per-agent shared-context prompts, and enforces feasibility through action-mask validation.

The reported results cover Objective 1 and Objective 2. Objective 1 provides the current data for planning-architecture and prompt-format comparisons under the shelf-delivery objective. Objective 2 provides the current data for call-aware control, including delivery totals, calls per delivery, seconds per delivery, and the model-size comparison for the shared-context natural-language session.

Within the available data, Objective 1 reports more deliveries for shared-context planning than for centralized planning in the matched natural-language comparison, and more deliveries for natural-language prompting than for JSON prompting in the matched shared-context comparison. Objective 2 reports 303 deliveries for the completed natural-language shared-context session and 53 deliveries for the completed JSON shared-context session. In the completed natural-language model comparison for Objective 2, `llama3.2:3b` produced the highest total delivery count.

Completed aggregate results are not yet available for Objective 3, the battery-aware objective. This objective remains part of the defined evaluation framework, but its aggregate outcome is not yet reported in the current results chapter.

5.1 Future Work

- **Structured outputs and tighter parsing:** require the model to output a strict schema (e.g., {`action_id, hold_steps, rationale`}) and reject non-conforming responses to reduce ambiguity.
- **Centralized versus decentralized planning:** compare per-agent prompts (current approach) with a centralized planner that outputs a joint action vector, including explicit coordination constraints.
- **Candidate-set design:** learn or compute compact candidate action sets (e.g., nearest requested shelves, relevant support targets for pickers) while guaranteeing that necessary actions remain available.

- **Multi-seed and scaling studies:** extend evaluation to many random seeds and larger layouts (more shelves, more agents) to quantify robustness and scalability.
- **MARL baselines:** train/evaluate proximal policy optimization (PPO) or other MARL methods on Battery-TA-RWARE and compare to LM in throughput, generalization, and compute cost.
- **Cost-aware deployment:** measure inference latency, token usage, and energy cost across model sizes to assess feasibility for embedded or real-time settings.

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[2] T. Arai, E. Pagello, and L. E. Parker, "Editorial: Advances in multi-robot systems," *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 655–661, 2002.

[3] Í. R. d. C. Barros and T. P. d. Nascimento, "Robotic mobile fulfillment systems: A survey on recent developments and research opportunities," *Robotics and Autonomous Systems*, vol. 137, p. 103729, 2021.

[4] V. Bhat, A. U. Kaypak, P. Krishnamurthy, R. Karri, and F. Khorrami, "Grounding large language models for robot task planning using closed-loop state feedback," 2025, received: 27 April 2025; revised: 10 October 2025; accepted: 17 October 2025.

[5] Y. U. Cao, A. S. Fukunaga, and A. B. Kahng, "Cooperative mobile robotics: Antecedents and directions," *Autonomous Robots*, vol. 4, no. 1, pp. 7–27, 1997.

[6] H. H. Clark, *Using Language*. Cambridge, UK: Cambridge University Press, 1996.

[7] A. Dawane, "On-device small language model inference acceleration for real-time robotics: A survey," in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, 2026.

[8] R. Dunbar, *Grooming, Gossip, and the Evolution of Language*. Cambridge, MA, USA: Harvard University Press, 1996.

[9] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[10] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *arXiv preprint arXiv:2201.07207*, 2022.

[11] Y. Huang, "Collaborative multi-robot task planning using large language models," Master's thesis, Uppsala University, 2025, accessed 2026-02-26. [Online]. Available: https://uu.diva-portal.org/smash/get/diva2:2023514/FULLTEXT01.pdf

[12] R. Karam, A. A. Nguyen, R. Lin, D. R. Martin, D. Morales, B. A. Butler, and M. Egerstedt, "Collaboration in multi-robot systems: Taxonomy and survey over frameworks for collaboration," 2026.

[13] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. K. To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 677–684.

[14] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. Wing Keung To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," 2022.

[15] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding in large-scale warehouses," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 13, 2021, pp. 11 272–11 281.

[16] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299, accessed 2026-02-26. [Online]. Available: https://ieeexplore.ieee.org/document/10610855

[17] X. Niu, N. C. Barajas, and D. G. Broo, "Enabling symbiosis in multi-robot systems through multi-agent reinforcement learning," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–7.

[18] X. Niu and D. G. Broo, "Investigating symbiosis in robotic ecosystems: A case study for multi-robot reinforcement learning reward shaping," in *2025 9th International Conference on Robotics and Automation Sciences (ICRAS)*, 2025, pp. 112–117.

[19] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," 2020, accessed 2026-02-26. [Online]. Available: https://arxiv.org/abs/2006.07869

[20] M. A. Sebok and H. G. Tanner, "Cooperative planning for physically interacting heterogeneous robots," *Frontiers in Robotics and AI*, vol. 11, p. 1172105,

2024. [Online]. Available: https://www.frontiersin.org/articles/10.3389/frobt.2024.1172105/full

[21] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.

[22] M. Tomasello, *Origins of Human Communication*. Cambridge, MA, USA: MIT Press, 2008.

[23] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," *AI Magazine*, vol. 29, no. 1, pp. 9–20, 2008.

[24] L. Xiao and T. Yamasaki, "Probing prompt design for socially compliant robot navigation with vision-language models," 2026, accessed 2026-02-26.

[25] Z. Yan, N. Jouandeau, and A. A. Cherif, "A survey and analysis of multi-robot coordination," *International Journal of Advanced Robotic Systems*, vol. 10, no. 12, 2013, accessed 2026-02-26. [Online]. Available: https://journals.sagepub.com/doi/full/10.5772/57313

[26] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[27] C. Zhang and V. Lesser, "Coordinating multi-agent reinforcement learning with limited communication," in *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2013, accessed 2026-02-26. [Online]. Available: https://www.ifaamas.org/Proceedings/aamas2013/docs/p1101.pdf

A Appendix

A.1 Reproducibility: Running the Controllers

The experiments in this thesis are executed from the Battery-TA-RWARE repository in this workspace. Example commands:

```
python Battery-TA-RWARE/scripts/run_llm.py \
  --env_id tarware-tiny-1agvs-1pickers-partialobs-chg-v1 \
  --seed 0 \
  --num_episodes 1 \
  --model llama3.2:3b
```

Listing 1: Run the basic per-agent language-model controller for one episode.

```
python Battery-TA-RWARE/scripts/run_centralized_llm_experiments.py \
  --prompt_format language \
  --seed 0
```

Listing 2: Run the centralized shelf-delivery experiments.

```
python Battery-TA-RWARE/scripts/run_shared_context_llm_expt.py \
  --prompt_format language \
  --seed 0
```

Listing 3: Run the shared-context shelf-delivery experiments.

```
python Battery-TA-RWARE/scripts/run_obj2_shared_context_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 4: Run the Objective 2 shared-context call-aware experiments.

```
python Battery-TA-RWARE/scripts/run_heuristic.py \
  --env_id tarware-tiny-1agvs-1pickers-partialobs-chg-v1 \
  --seed 0 \
  --num_episodes 1
```

Listing 5: Run the heuristic policy for one episode.

The main result folders discussed in the thesis are:

- results/shelf/central_llm_experiments: Objective 1, centralized planning.
- results/shelf/shared_context_llm_experiments: Objective 1, shared-context per-agent planning.
- results/calls/shared_context: Objective 2, shared-context per-agent planning with call-efficiency objective.
- results/battery: planned battery-oriented experiments (not yet populated with completed aggregate sessions at the time of writing).

A.2 Prompt Excerpt (Illustrative)

The LM is prompted with an explicit mapping from relevant state elements to discrete action identifiers. The exact prompt depends on the experiment family: centralized prompts describe all agents at once, while shared-context prompts describe one current agent together with selected global context. The full prompts are generated programmatically; the excerpt below illustrates the style (line breaks and ordering may vary):

```
Current global state:
- Requested shelves:
  - Requested shelf 24 is at position (11,5). To go there, choose
    action id 50.
  - Requested shelf 39 is at position (6,8). To go there, choose
    action id 29.
...
You are agent_0 (type=AGV).
Choose exactly one action_id from the candidate list.
Output format: action_id=<integer>
```

Listing 6: Illustrative prompt excerpt.

A.3 Metric Definitions

- **Total shelf deliveries:** number of requested shelves delivered to goal locations during an episode.
- **Overall pick rate:** deliveries/hour computed as

$$\text{pick\_rate} = \frac{\text{deliveries} \cdot 3600}{5 \cdot \text{episode\_steps}},$$

where 5 steps/second is the environment's time scale used by the logging script.

- **Clashes and stuck events:** environment-defined counts related to collision resolution and deadlock-like behaviour.
- **Global episode return:** sum of environment rewards across all agents and time steps.